Reviewer Pack — Minimal Root System Length and Communication Complexity Rank…

Entry e4a667958de4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 04:48:54 UTC

Minimal Root System Length and Communication Complexity Rank Variance Correlation

Entry ID: `e4a667958de4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 04:48:54 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Root Systems) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every CNF formula φ with n variables, the minimal root system length of its associated representation in the Euclidean space is linearly correlated with its communication complexity rank variance, such that |L(φ)| = Θ(w(φ)^2), where L(φ) is the minimal root system length and w(φ) is the communication complexity rank variance.

Rationale (proposer's reasoning):

> Root systems are a fundamental tool in Lie theory and have been used to study symmetries in geometry. Communication complexity, being about information exchange between parties, has a geometric interpretation when seen as the distance between sets of possible computations. The conjecture bridges these fields by suggesting that the structure of root systems can be exploited to understand communication complexity.

Taxonomy category: `LieTheoryCommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `009c3c3c414ed11e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across at least 24 out of 30 random seeds, the Pearson correlation coefficient between minimal root system length |L(φ)| and communication complexity rank variance w(φ) is greater than or equal to 0.8 and less than or equal to 1. The conjecture is falsified if any seed produces a correlation coefficient outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'root system length' AND 'communication complexity rank variance' - 'Lie theory' AND 'matrix rank in communication complexity' - 'Euclidean space representation' AND 'CNF formula communication complexity'

Top relevant hits considered: - [http://arxiv.org/abs/1409.0184v4] Prenilpotent pairs in the E10 root system - [http://arxiv.org/abs/1601.01857v2] Cohomology of Complements of Toric Arrangements Associated to Root Systems - [http://arxiv.org/abs/2504.02827v1] On Vanishing Variance in Transformer Length Generalization - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2108.00651v1] Curvature of matrix and reductive Lie groups - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/0804.4433v1] SPACE: the SPectroscopic All-sky Cosmic Explorer - [http://arxiv.org/abs/0804.3505v3] A geometric study of Wasserstein spaces: Euclidean spaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(abs(c) != abs(d) for c, d in zip(clause, clause[1:])):
                clauses.append(clause)
        return clauses

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
```

```
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = Fraction(1, matrix[i][i])
        for j in range(n):
            matrix[i][j] *= factor
        for j in range(n):
            if i != j:
                factor = -matrix[j][i]
                for k in range(n):
                    matrix[j][k] += factor * matrix[i][k]
    return matrix

def rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if all(abs(x) < 1e-9 for x in matrix[i]):
            continue
        rank += 1
        for j in range(n):
            if i != j:
                factor = -matrix[j][i]
                for k in range(n):
                    matrix[j][k] += factor * matrix[i][k]
    return rank

def communication_complexity_rank_variance(clauses, n):
    m = len(clauses)
    A = [[0] * (m + 1) for _ in range(m + 1)]
    for i in range(m):
        for j in range(i+1, m):
            if any(abs(c) == abs(d) for c, d in zip(clauses[i], clauses[j])):
                A[i][j] = A[j][i] = 1
    A[m][m] = 1
    A = gaussian_elimination(A)
    return rank(A)

def minimal_root_system_length(n):
    # Placeholder function for the actual calculation
    # This is a dummy implementation and should be replaced with the actual formula
    return n

instances_tested = 0
total_L = 0
total_w = 0
max_n = 0

for n in [5, 10, 15, 20, 30, 40]:
    if n > max_n:
        max_n = n

    for _ in range(5):
        cnf = generate_cnf(n)
        L = minimal_root_system_length(n)
        w = communication_complexity_rank_variance(cnf, n)
```

```
            total_L += L
            total_w += w
            instances_tested += 1

    mean_L = total_L / instances_tested
    mean_w = total_w / instances_tested
    correlation_coefficient = (instances_tested * sum(L * w for L, w in zip([mean_L] * instances_t
                               instances_tested * mean_L * mean_w) / math.sqrt((instances_tested *
                                                                   (instances_tested * sum(w**2 for

    conjecture_holds = 0.8 <= correlation_coefficient <= 1
    counterexample = "" if conjecture_holds else "correlation_outside_range"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_outside_range\" first_failing_seed=
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot verify the correlation coefficient for the conjecture. | next: Re-run the test with a longer timeout or increase system resources to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14980
2	preregistration	ollama_remote	glm4:latest	0	0	10605
3	novelty	ollama_remote	glm4:latest	0	0	9737
4	novelty	ollama_remote	glm4:latest	0	0	10092
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45514
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10189
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13305
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15177
9	judge	ollama_remote	glm4:latest	0	0	16704

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 146303 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e4a667958de4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e4a667958de4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e4a667958de4.tar.gz` (if generated)